Fabricando patos

Original: *Head First Design Patterns*, editorial **O'Reilly**

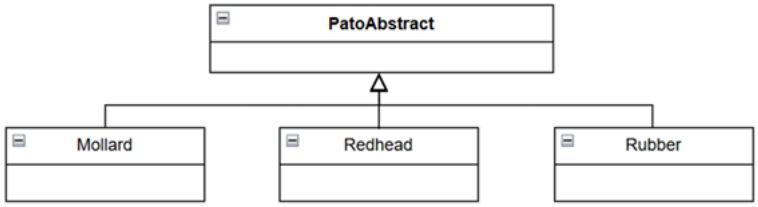
Volvemos con los **Patos**. Resulta que estás dando vueltas al código que has estado haciendo sobre tu granja de Patos, y te ha surgido una duda. Cada vez que haces un 'new' estás instanciando una clase concreta. Pero cada vez que lo haces, tu programa se vuelve más frágil y menos flexible.

```
Mallard pato = new Mallard ();
```

Esto NO es incorrecto. Pero estás haciendo que pato dependa 100% de la instanciación de una clase concreta (Mallard). Así es como hemos trabajado hasta ahora. E insisto, no es un error. Pero esto hace que tu código sea poco flexible. Y **acoplado**. La variable sólo puede trabajar con Mallards.

¿Y si mi código tiene que trabajar con varios tipos de patos?

Bueno, supongamos esta estructura que de herencia que ya conocíamos:



Con nuestros conocimientos, sabemos que una solución podría ser:

```
PatoAbstract pato = new Mallard ();
```

Podemos decidir que pato sea de tipo abstracto. Esto permitiría instanciar cualquier tipo concreto de clase y manejarla con la misma variable.

```
PatoAbstract pato;

if (opcion == 0) {
    pato = new Mallard();
} else if (opcion == 1) {
    pato = new RedHead();
} else if (opcion == 2) {
    pato = new Rubber();
}
```

Haciendo esto, hemos conseguido que varias clases diferentes puedan ser instanciadas, pero la decisión de cuál se va a instanciar se hace en **tiempo de ejecución**, dependiendo de una serie de condiciones (opción).

¿Qué te parece la solución?

Pues está muy bien, funcionar funciona… salvo que, cuando tienes que hacer extensiones, cambiar el código, arreglar bugs, etc. estás obligado a reabrir este código y retocarlo. Y sucede que, este tipo de código, a veces está disperso por toda la aplicación, lo que complica el mantenimiento.

Por tanto, sería lógico pensar que es mejor centralizar la instanciación de objetos. Cómo hacerlo es lo que vamos a ver en este documento.

Un nuevo enfoque

Digamos que, tu idea general para un único Pato es esta:

```
private Pato despegar() {
    Pato ret = new Pato();

    ret.prepararPista();
    ret.acelerar();
    ret.volar();

    return ret;
}
```

Pero obviamente, tenemos varios tipos diferentes de Patos, por lo tanto, nos basaremos en el ejemplo anterior de los if para hacer algo similar.

```
private PatoAbstract despegar(String type) {
    PatoAbstract ret = null;

    if (type.equals("Mallard")) {
        ret = new Mallard();
    } else if (type.equals("RedHead")) {
        ret = new RedHead();
    } else if (type.equals("Rubber")) {
        ret = new Rubber();
    }

    ret.prepararPista();
    ret.acelerar();
    ret.volar();

    return ret;
}
```

Sencillo. Y has **desacoplado** la creación de patos. Sin embargo… esta solución no es 100% ferpecta. Acarrea varias pegas cuando empezamos a añadir tipos de patos diferentes.

```
private PatoAbstract despegar(String type) {
    PatoAbstract ret = null;

    if (type.equals("Mallard")) {
        ret = new Mallard();
    } else if (type.equals("RedHead")) {
        ret = new RedHead();
    } else if (type.equals("Rubber")) {
        ret = new Rubber();
    } else if (type.equals("RouenS")) {
        ret = new Rubber();
    }else if (type.equals("Saxony")) {
        ret = new Rubber();
    }

    ret.prepararPista();
    ret.acelerar();
    ret.volar();

    return ret;
}
```

Si te fijas, la parte del if se corresponde con una parte del código que puede cambiar con el tiempo. Simplemente, porque añadimos un tipo de pato nuevo. En cambio, la parte de preparar la pista, acelerar y volar, no cambiará nunca.

También, recordemos lo que dijo un sabio:

"Tu código debe ser abierto para la extensión, pero cerrado para la modificación"

Obviamente, este código permite hacer extensiones. Con hacer un simple public class Icelandic extends Saxony {} y añadir esta variedad al método despegar (), ya tenemos un pato nuevo en nuestro programa.

PERO. Cada vez que añadamos un nuevo tipo de pato, o lo eliminemos, o lo que sea; va a ser necesario cambiar el código de despegar (). Luego esta solución NO es cerrada para el cambio.

Luego, debería de haber una solución mejor. ¿No?

Encapsulando la creación de objetos

Vamos a quitarnos la parte que cambia de en medio, y **encapsularla**. Nadie va a tener acceso a cómo va a estar implementada esa parte, pero otros objetos van a saber cómo usarla.

```
private PatoAbstract despegar(String type) {
    PatoAbstract ret = null;

    // DELETE //

    ret.prepararPista();
    ret.acelerar();
    ret.volar();

    return ret;
}
```

Y lo vamos a meter en una Fábrica de Patos, una nueva clase experta en fabricar patos de alta calidad.

```
public class FactoriaDePatosSencilla (){

    public PatoAbstract damePato (String type) {
        PatoAbstract ret = null;

        if (type.equals("Mallard")) {
            ret = new Mallard();
        } else if (type.equals("RedHead")) {
            ret = new RedHead();
        } else if (type.equals("Rubber")) {
            ret = new Rubber();
        } else if (type.equals("Rouens")) {
            ret = new Rubber();
        }else if (type.equals("Saxony")) {
            ret = new Rubber();
        }

        return ret;
    }
}
```

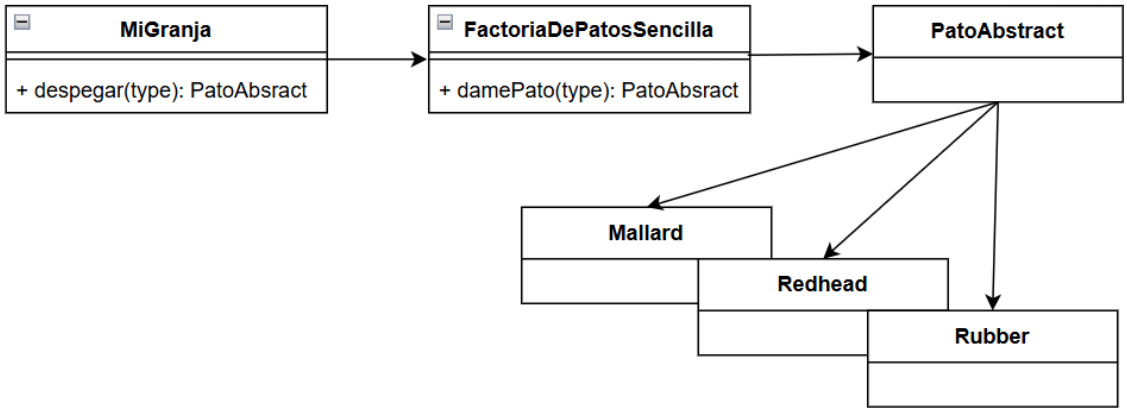
Añora, sólo nos queda retocar el código de despegar ():

```
private FactoriaDePatosSencilla factoria = new FactoriaDePatosSencilla ();

private PatoAbstract despegar(String type) {
    PatoAbstract ret = null;

    ret = factoria.damePato(type);

    ret.prepararPista();
    ret.acelerar();
    ret.volar();

    return ret;
}
```

De esta forma, conseguimos delegar en Factoría de Patos Sencilla toda la responsabilidad de crear los patos (objetos). Y, además, esto permite tener un programa cerrado al cambio. A partir de ahora, cada vez que haya que añadir un tipo de pato nuevo, simplemente añadimos una nueva clase pato y **modificamos** exclusivamente **la Factoría** sin afectar al resto del programa.

Complicándonos la vida

Esto que acabas de ver se llama **Simple Factory** y no se considera un verdadero Patrón de Diseño. Pero se usa mucho, por lo tanto, no es mala idea conocerlo. El diagrama de clases de nuestra fábrica de patos por tanto nos quedaría más o menos así usando un Simple Factory.



Esta solución es correcta para cuando tengo una única granja de patos. Pero resulta que, nuestro jefe ha dicho que tenemos **tres granjas** en diferentes lugares, y cada una de ellas fabrica un tipo de pato diferente. La granja de Bilbao sólo tiene Mallards y Redheads; la de Vitoria solamente Rubbers; y la de San Sebastián tiene Mallards, Rouens y Saxxony.

Puede que lo primero que se te ocurra es quitar **FactoríaDePatosSencilla** y crear tres factorías para cada una de las ciudades. Entonces, te quedaría algo así como:

```
FactoriaDePatosBibao factoriaBilbao = new FactoriaDePatosBibao ();
MiGranja granjaBilbao = new MiGranja (factoriaBilbao);
granjaBilbao.damePato("Mallard");

FactoriaDePatosVitoria factoriaVitoria = new FactoriaDePatosVitoria ();
MiGranja granjaVitoria = new MiGranja (factoriaVitoria);
granjaVitoria.damePato("Rubber");
```

Esto es, creamos una Factoría para cada ciudad con los patos que puede producir, luego pasamos a MiGranja una referencia con esa Factoría; lo que nos permite entonces crear los patos de esa factoría. Ya podemos despegar ().

¿Guay? ¿Funciona? Hemos resuelto el problema, ¿cierto?

Pues sí. Y no. Como pasa siempre, cuando diseñamos un proyecto, hay que pensar en el largo plazo. Suponte que cada factoría de cada ciudad hace los patos de forma distinta, aunque sean del mismo tipo. A lo mejor, los Mallards de Bilbao preparan la pista (prepararPista ()) de forma ligeramente diferente porque hay fuertes vientos del oeste, cosa que no ocurre en San Sebastián.

Si pretendemos tener un framework que agrupe a todas las ciudades y la creación de patos, pero permitiendo que todo el tinglado sea flexible, hay que cambiar la cosa.

El Framework de las fábricas de patos (Factory Method)

Existe una forma de localizar la creación de patos en MiGranja Y A LA VEZ permitir que cada ciudad haga las cosas a su manera.

Vamos a devolver el método de damePato () a MiGranja, pero como un método abstracto, y entonces, crear una subclase de Granja para cada ciudad.

```
public abstract class MiGranja {

    public PatoAbstract despegar (String type) {
        PatoAbstract ret = null;

        ret = damePato(type);

        ret.prepararPista();
        ret.acelerar();
        ret.volar();

        return ret;
    }

    protected abstract PatoAbstract damePato (String type);
}
```

En principio, todo se ve igual. Lo único que hemos hecho es desplazar el método factoría damePato (). La clase está ahora preparada para hacer extensiones, una por cada ciudad.

Por ejemplo, podríamos hacer:

```
public class MiGranjaDeBilbao extends MiGranja{

    @Override
    protected PatoAbstract damePato(String type) {
        PatoAbstract ret = null;

        if (type.equals("Mallard")) {
            ret = new Mallard();
        } else if (type.equals("RedHead")) {
            ret = new RedHead();
        }

        return ret;
    }

}
```

Y también:

```
public class MiGranjaDeVitoria extends MiGranja{

    @Override
    protected PatoAbstract damePato(String type) {
        PatoAbstract ret = null;

        if (type.equals("Rubber")) {
            ret = new Rubber();
        }

        return ret;
    }

}
```

Y también:

```
public class MiGranjaDeSanSebastian extends MiGranja {

    @Override
    protected PatoAbstract damePato(String type) {
        PatoAbstract ret = null;

        if (type.equals("Mallard")) {
            ret = new Mallard();
        } else if (type.equals("Rouens")) {
            ret = new Rouens();
        } else if (type.equals("Saxxony")) {
            ret = new Saxxony();
        }

        return ret;
    }

}
```

¿Por qué esto es mejor? Bien… porque desde el punto de vista de MiGranja, el método damePato () es abstracto. La implementación concreta de lo que hace está en una subclase. **No sabe** exactamente cuál. A demás, el método devuelve un pato abstracto… por tanto, MiGranja **no tiene ni idea** de exactamente el pato que está devolviendo.

La gracia de esta Factoría de Métodos es que estás delegando en subclases como MiGranjaDeBilbao todo lo referente a la fabricación de cada pato. Fíjate en el código del método despegar () de MiGranja. Vamos a pedir un Mallard de Bilbao y un Mallard de San Sebastián.

```
MiGranjaDeBilbao granjaBilbao = new MiGranjaDeBilbao ();
granjaBilbao.damePato("Mallard");

MiGranjaDeVitoria granjaVitoria = new MiGranjaDeVitoria ();
granjaVitoria.damePato("Mallard");
```

Dado que la instanciación de Mallard se realiza en cada una de las clases Granja, podemos añadir las peculiaridades de cada Mallard en nuestro proyecto. Por ejemplo, vamos a ponerle al Mallard de Bilbao la habilidad de controlar los fuertes vientos del oeste:

Dado este pato abstracto:

```
public abstract class PatoAbstract {

    public abstract void volar();

    public abstract void prepararPista();

    public abstract void acelerar();

}
```

Podemos hacer:

```
public class PatoMallardBilbao extends PatoAbstract {

    @Override
    public void volar() {

    }

    @Override
    public void prepararPista() {
        // Código para controlar vientos fuertes del oeste
    }

    @Override
    public void acelerar() {

    }

}
```

Por lo que, con este simple cambio:

```
public class MiGranjaDeBilbao extends MiGranja{

    @Override
    protected PatoAbstract damePato(String type) {
        PatoAbstract ret = null;

        if (type.equals("Mallard")) {
            ret = new MallardBilbao();
        } else if (type.equals("RedHead")) {
            ret = new RedHead();
        }

        return ret;
    }

}
```

Tenemos ya una granja en Bilbao que genera Mallards, pero el tipo específico de Mallards especiales que requieren controlar el viento del oeste. Y lo hemos conseguido delegando la responsabilidad a la subclase, sin tener que tocar apenas una línea de código, precisamente, en la clase cuya responsabilidad es la de instanciar objetos.

Piensa ahora lo sencillo que sería añadir una fábrica de patos en Pamplona, un lugar conocido por sus numerosos y diversos patos…

Ejercicio Propuesto

Recrea el ejemplo mencionado con las fábricas de patos en Bilbao, Vitoria y San Sebastián. Crea una Granja en Pamplona. Añade algunos patos particulares de la ciudad.

Modifica los métodos prepararPista (), acelerar () y volar () de cada Pato para que escriba una línea similar a esta: “Mallard de Bilbao – volando ()”.

Comprueba la ejecución de tu factoría de métodos.